INFRAESTRUCTURA DE SERVICIOS

Práctica individual de Docker Compose con cinco servicios

Sistema de Reservación de Espacios

Frontend (Nginx) · Backend (PHP + Apache) · MariaDB · Redis · Mailpit

Estudiante	Sandra Chávez Hernández
Número de control	22121358
Grupo	A
Asignatura	Infraestructura de Servicios
Modalidad	Individual
Sistema asignado	Reservación de espacios (último dígito 8)
Mejora individual	Filtro de búsqueda
Fecha	11 de octubre de 2026

Contenido

1. Introducción y objetivos
2. Dominio asignado y modelo de datos
3. Arquitectura de la solución
4. Estructura del repositorio
5. Desarrollo de la práctica por etapas
6. Integración y puesta en marcha
7. Pruebas funcionales del CRUD
8. Persistencia en MariaDB
9. Redis: caché, invalidación y contador
10. Notificaciones con Mailpit
11. Red, aislamiento y recuperación
12. Resolución de nombres por DNS interno
13. Seguridad
14. Mejora individual: filtro de búsqueda
15. Resumen de pruebas obligatorias
16. Notas técnicas
17. Preguntas de reflexión
18. Lista de cotejo
19. Conclusiones

1. Introducción y objetivos

En esta práctica se diseñó, construyó y verificó una aplicación web distribuida en **cinco contenedores** orquestados con **Docker Compose**. La aplicación es un sistema de reservación de espacios. Permite registrar, consultar, modificar y eliminar reservaciones de auditorios, laboratorios y salas.

Los cinco servicios se comunican por una red interna llamada `app_network` y se encuentran entre sí por su nombre de servicio. MariaDB conserva los datos en un volumen nombrado. Redis guarda en caché el listado y cuenta las consultas. Mailpit captura un correo de prueba cada vez que se crea una reservación.

Objetivos cumplidos:

- Se construyeron imágenes propias para el frontend y el backend mediante Dockerfile.
- Se definieron y ejecutaron los cinco servicios desde un único archivo `compose.yaml`.
- Se configuró una red interna y se usaron los nombres de servicio como host.
- La información de MariaDB se conserva en un volumen nombrado.
- Redis funciona como caché con TTL y como contador de consultas.
- El backend genera una notificación de correo que se puede ver en Mailpit.
- Se aplicaron health checks, `depends_on`, variables de entorno y política de reinicio.
- La solución se verificó con comandos, registros y pruebas funcionales.

2. Dominio asignado y modelo de datos

El último dígito del número de control (22121358) es **8**. Según la tabla de asignación, el sistema es **Reservación de espacios** y la entidad principal es **Reservación**. Cada reservación indica qué espacio se reserva, quién lo solicita, su correo, la fecha, el horario y el estado (pendiente, confirmada o cancelada).

Columna	Tipo	Restricción
id	INT UNSIGNED AUTO_INCREMENT	PRIMARY KEY
espacio	VARCHAR(100)	NOT NULL, CHECK longitud ≥ 3
solicitante	VARCHAR(100)	NOT NULL, CHECK longitud ≥ 3
correo	VARCHAR(150)	NOT NULL
fecha	DATE	NOT NULL
hora_inicio / hora_fin	TIME	NOT NULL, CHECK hora_fin > hora_inicio
estado	ENUM('pendiente','confirmada','cancelada')	DEFAULT 'pendiente'
creado_en / actualizado_en	TIMESTAMP	Automáticos

Además, la restricción `UNIQUE (espacio, fecha, hora_inicio)` impide que el mismo espacio se reserve dos veces con la misma fecha y hora de inicio. Si se intenta, la API responde **409 Conflict**. El script `database/init.sql` crea la tabla e inserta tres registros iniciales:

```
-- Sistema de reservación de espacios
-- Se ejecuta solo la primera vez que se crea el volumen de datos.

CREATE TABLE IF NOT EXISTS reservaciones (
    id           INT UNSIGNED NOT NULL AUTO_INCREMENT,
    espacio      VARCHAR(100) NOT NULL,
    solicitante  VARCHAR(100) NOT NULL,
    correo       VARCHAR(150) NOT NULL,
    fecha        DATE         NOT NULL,
    hora_inicio  TIME         NOT NULL,
    hora_fin     TIME         NOT NULL,
    estado       ENUM('pendiente', 'confirmada', 'cancelada') NOT NULL DEFAULT 'pendiente',
    creado_en    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    actualizado_en TIMESTAMP  NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT pk_reservaciones PRIMARY KEY (id),
    CONSTRAINT uq_espacio_horario UNIQUE (espacio, fecha, hora_inicio),
    CONSTRAINT chk_horario CHECK (hora_fin > hora_inicio),
    CONSTRAINT chk_espacio CHECK (CHAR_LENGTH(TRIM(espacio)) >= 3),
    CONSTRAINT chk_solicitante CHECK (CHAR_LENGTH(TRIM(solicitante)) >= 3)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

INSERT INTO reservaciones (espacio, solicitante, correo, fecha, hora_inicio, hora_fin, estado)
VALUES
    ('Laboratorio de Cómputo 1', 'Ana López',     'ana.lopez@practica.local',     '2026-10-15',
'08:00', '10:00', 'confirmada'),
    ('Auditorio Principal',      'Carlos Pérez',  'carlos.perez@practica.local',  '2026-10-16',
'12:00', '14:00', 'pendiente'),
    ('Sala de Juntas A',         'María García',  'maria.garcia@practica.local',  '2026-10-17',
'16:00', '17:30', 'cancelada');
```

3. Arquitectura de la solución

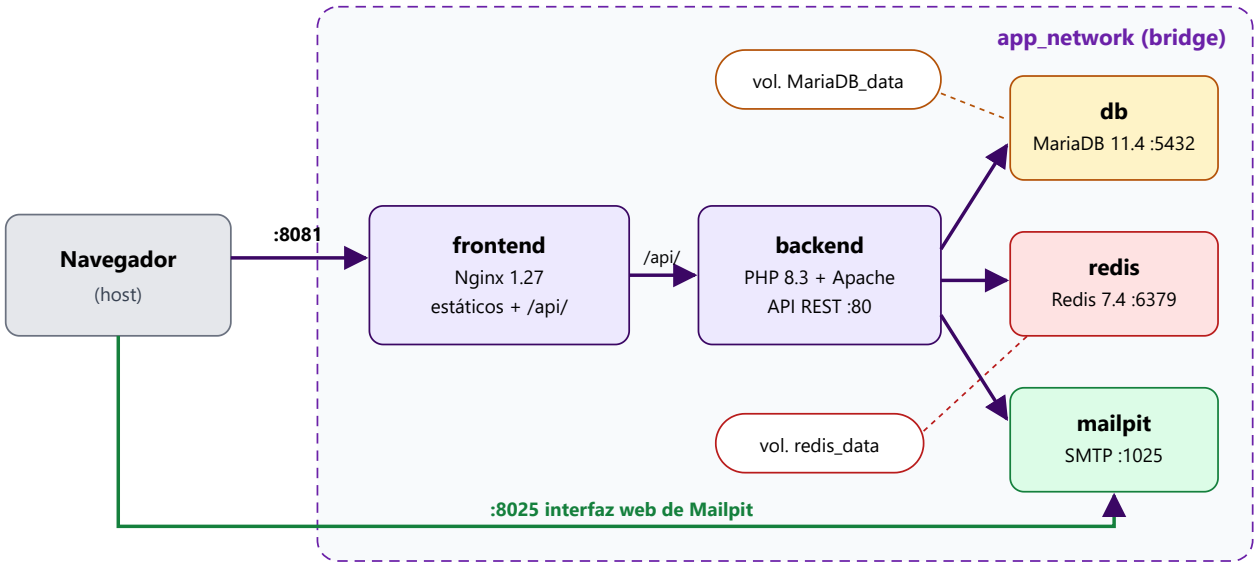


Figura 1. Arquitectura de los cinco servicios. Solo se publican los puertos 8081 y 8025.

Servicio	Tecnología	Responsabilidad	Exposición
frontend	nginx:1.27-alpine + HTML/CSS/JS	Interfaz y proxy de `/api/` hacia backend	Host 8081
backend	php:8.3-apache (imagen propia)	API REST, lógica, MariaDB, Redis y SMTP	Solo red interna
db	mariadb:11.4	Persistencia de reservaciones	Solo red interna
redis	redis:7.4-alpine	Caché (TTL 60 s) y contador	Solo red interna
mailpit	axllent/mailpit:v1.21.8	Captura de correos de prueba	Interfaz web 8025

Flujo: el navegador abre `http://localhost:8081`. Nginx entrega los archivos estáticos y reenvía las peticiones `/api/*` a `http://backend:80`. El backend consulta `db:5432`, usa `redis:6379` para la caché y el contador, y envía correos a `mailpit:1025`. Desde el host, la interfaz de Mailpit está en `http://localhost:8025`.

4. Estructura del repositorio

```
proyecto-chavez-sandra/
├── compose.yaml           # definición de los cinco servicios
├── .env.example           # plantilla de variables sin contraseñas reales
├── .gitignore             # excluye .env
├── README.md              # documentación y pasos para reproducir
├── database/
│   └── init.sql           # tabla, restricciones y 3 registros iniciales
├── backend/
│   ├── Dockerfile         # php:8.3-apache + pdo_mysql + phpredis
│   └── src/
│       ├── index.php      # router y endpoints REST
│       └── lib/           # Database.php, Cache.php, Mailer.php
├── frontend/
│   ├── Dockerfile         # nginx:1.27-alpine
│   ├── nginx.conf         # estáticos + proxy /api/ → backend
│   └── src/               # index.html, app.js, styles.css
└── informe/               # este reporte y sus capturas
```

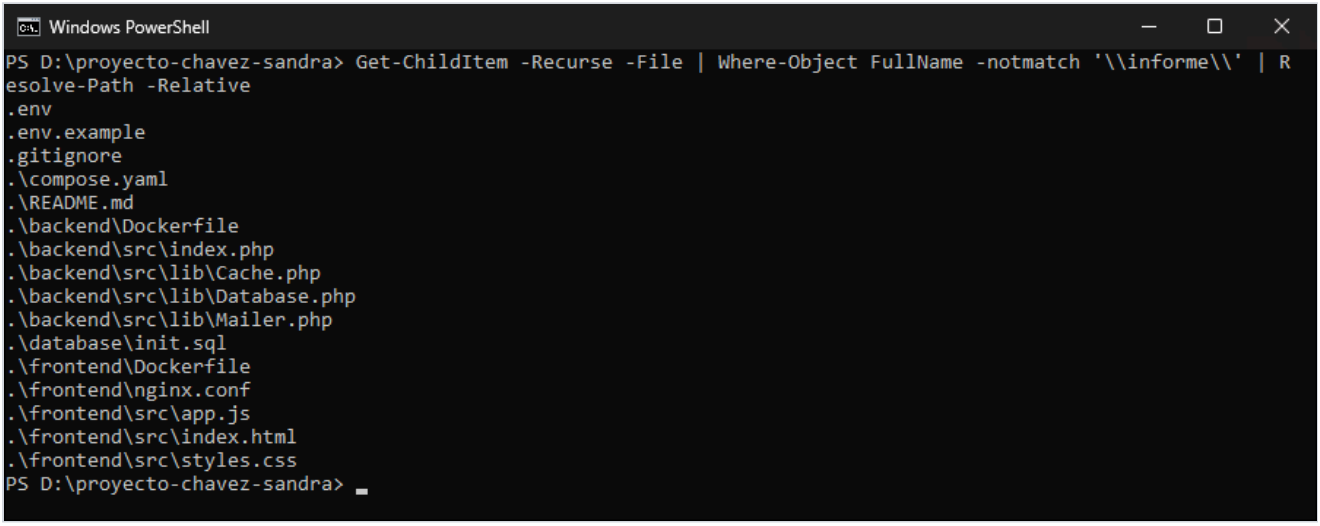


Figura 2. Archivos del proyecto. El archivo `.env` existe solo localmente y no se versiona.

5. Desarrollo de la práctica por etapas

Etapa 1. Preparación

Se creó la estructura de directorios y el archivo `.gitignore`, que excluye `.env`. También se creó `.env.example` con valores de ejemplo. El archivo `.env` local tiene contraseñas generadas aleatoriamente. Se eligió el dominio asignado y se diseñó `init.sql` con una tabla, una llave primaria, restricciones y tres registros iniciales (sección 2).

Archivo compose.yaml

Este es el archivo completo que define los cinco servicios, la red `app_network`, los volúmenes, los health checks, las dependencias y la política `restart: unless-stopped`:

```
# Práctica individual: Docker Compose con cinco servicios
# Sistema asignado: Reservación de espacios (último dígito 8)
name: reservaciones

services:
  frontend:
    build: ./frontend
    restart: unless-stopped
    ports:
      - "${FRONTEND_PORT:-8081}:80"   # único acceso público a la aplicación (8081 por defecto)
    depends_on:
      backend:
        condition: service_healthy
    networks:
      - app_network

  backend:
    build: ./backend
    restart: unless-stopped
    expose:
      - "80"                 # solo red interna, sin puerto en el host
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: ${DB_NAME}
      DB_USER: ${DB_USER}
      DB_PASSWORD: ${DB_PASSWORD}
      REDIS_HOST: redis
      REDIS_PORT: 6379
      REDIS_PASSWORD: ${REDIS_PASSWORD}
      CACHE_TTL: ${CACHE_TTL:-60}
      SMTP_HOST: mailpit
      SMTP_PORT: 1025
      MAIL_FROM: ${MAIL_FROM:-reservaciones@practica.local}
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
      mailpit:
        condition: service_started
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost/health"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```
      start_period: 10s
    networks:
      - app_network

  db:
    image: mariadb:11.4
    restart: unless-stopped
    # MariaDB escucha por defecto en 3306; la práctica pide DB_PORT=5432
    command: ["--port=5432", "--character-set-server=utf8mb4", "--collation-
server=utf8mb4_unicode_ci"]
    environment:
      MARIADB_DATABASE: ${DB_NAME}
      MARIADB_USER: ${DB_USER}
      MARIADB_PASSWORD: ${DB_PASSWORD}
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
    volumes:
      - MariaDB_data:/var/lib/mysql      # directorio de datos de MariaDB
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "mariadb -h 127.0.0.1 -P 5432 -u\"$$MARIADB_USER\" -
p\"$$MARIADB_PASSWORD\" -e 'SELECT 1' \"$$MARIADB_DATABASE\""]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s
    networks:
      - app_network

  redis:
    image: redis:7.4-alpine
    restart: unless-stopped
    command: ["sh", "-c", "exec redis-server --appendonly yes --requirepass \"$$REDIS_PASSWORD\""]
    environment:
      REDIS_PASSWORD: ${REDIS_PASSWORD}
      REDISCLI_AUTH: ${REDIS_PASSWORD}   # permite usar redis-cli sin escribir -a
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD-SHELL", "redis-cli ping | grep -q PONG"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app_network

  mailpit:
    image: axllent/mailpit:v1.21.8
    restart: unless-stopped
    ports:
      - "8025:8025"          # interfaz web; el SMTP 1025 queda interno
    networks:
      - app_network

networks:
  app_network:
    driver: bridge

volumes:
  MariaDB_data:
  redis_data:
```

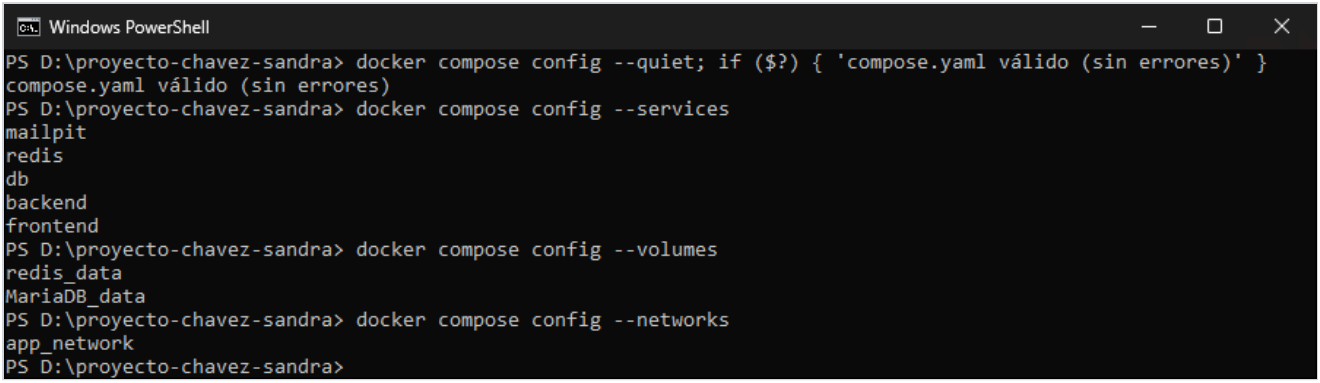


Figura 3. `docker compose config` valida el archivo sin errores. Se ven los cinco servicios, los dos volúmenes y la red.

Etapa 2. Base de datos y Redis

- **db:** usa la imagen `mariadb:11.4` con `--port=5432`. El volumen nombrado `MariaDB_data` se monta en el directorio de datos y `init.sql` se monta en `/docker-entrypoint-initdb.d/`. El health check ejecuta un `SELECT 1` por TCP con el usuario de la aplicación, así que solo pasa cuando la inicialización terminó.
- **redis:** usa la imagen `redis:7.4-alpine` con persistencia AOF (`--appendonly yes`), contraseña (`--requirepass`) y el volumen `redis_data`. El health check es `redis-cli ping`.

Etapa 3. Backend

El Dockerfile del backend parte de `php:8.3-apache` e instala las extensiones `pdo_mysql` (para MariaDB) y `redis` (phpredis, mediante PECL). La directiva `FallbackResource` envía todas las rutas a `index.php`, que funciona como router de la API.

```
FROM php:8.3-apache

# Extensiones: pdo_mysql para MariaDB y phpredis para Redis
RUN docker-php-ext-install pdo_mysql \
    && pecl install redis-6.1.0 \
    && docker-php-ext-enable redis \
    && rm -rf /tmp/pear

# Todas las rutas que no sean archivos se atienden con index.php (router de la API)
RUN echo 'FallbackResource /index.php' > /etc/apache2/conf-enabled/api-router.conf \
    && echo 'ServerName backend' > /etc/apache2/conf-enabled/servername.conf \
    && printf '<Directory /var/www/html/lib>\n  Require all denied\n</Directory>\n' >
/etc/apache2/conf-enabled/deny-lib.conf

COPY src/ /var/www/html/

EXPOSE 80
```

Método	Ruta	Descripción	Códigos
GET	/health	Verifica la conexión con MariaDB y Redis	200 / 503
GET	/reservaciones[?buscar=]	Listado con caché y contador	200 / 422
GET	/reservaciones/{id}	Detalle	200 / 404
POST	/reservaciones	Crea, invalida la caché y envía el correo	201 / 400 / 409 / 422
PUT	/reservaciones/{id}	Actualiza (también parcialmente)	200 / 404 / 409 / 422
DELETE	/reservaciones/{id}	Elimina e invalida la caché	204 / 404
GET	/estadisticas	Contador y TTL de la caché	200

La conexión usa las variables de entorno `DB_HOST=db`, `DB_PORT=5432`, `REDIS_HOST=redis`, `REDIS_PORT=6379`, `SMTP_HOST=mailpit` y `SMTP_PORT=1025`. Ninguna dirección es `localhost`. Este es el fragmento que implementa la caché y el contador en el listado:

```
$redis = redis_client();
$consultas = $redis->incr(CONTADOR);              // contador_consultas
$clave = cache_key($buscar);                       // reservaciones:lista
$cache = $redis->get($clave);
if ($cache !== false) {                            // HIT: respuesta desde Redis
    header('X-Cache: HIT');
    $registros = json_decode($cache, true);
} else {                                           // MISS: consulta a MariaDB
    header('X-Cache: MISS');
    $registros = db()->query('SELECT * FROM reservaciones ORDER BY fecha, hora_inicio')-
>fetchAll();
    $redis->setex($clave, cache_ttl(), json_encode($registros));   // TTL 60 s
}
```

Cada POST, PUT o DELETE llama a `cache_invalidar()`, que borra la clave del listado y las claves de búsquedas. Así la caché nunca entrega datos viejos después de un cambio.

Etapa 4. Mailpit

El backend incluye un cliente SMTP mínimo (`lib/Mailer.php`). Se conecta a `mailpit:1025` con `fsockopen` y envía los comandos EHLO, MAIL FROM, RCPT TO y DATA. Cuando se crea una reservación, envía un correo HTML con sus datos al correo del solicitante. Mailpit es solo una herramienta de desarrollo: captura el mensaje y lo muestra en su interfaz web, pero **no entrega correo real**.

Etapa 5. Frontend y proxy

El frontend es una página HTML, CSS y JavaScript servida por Nginx. El JavaScript usa la URL relativa `/api` y nunca `localhost`. Nginx reenvía `/api/` a `http://backend:80` y quita el prefijo. Se usa el DNS interno de Docker (`127.0.0.11`) con una variable para que Nginx vuelva a resolver el nombre `backend` si el contenedor se reinicia y cambia de IP.

```
server {
    listen 80;
    server_name _;

    # DNS interno de Docker: resuelve "backend" en cada petición,
    # así Nginx sigue funcionando si el contenedor del backend se reinicia y cambia de IP.
    resolver 127.0.0.11 valid=10s ipv6=off;
```

```
    root  /usr/share/nginx/html;
    index index.html;

    # Archivos estáticos
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Proxy de /api/ hacia el backend: /api/reservaciones -> http://backend/reservaciones
    location /api/ {
        set $backend_upstream http://backend:80;
        rewrite ^/api/(.*)$ /$1 break;
        proxy_pass $backend_upstream;

        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_read_timeout    15s;
    }
}
```

6. Integración y puesta en marcha

Para partir de cero, se eliminaron contenedores, volúmenes y red con `docker compose down -v`. Después se levantó todo con `docker compose up -d --build --wait`. La salida muestra el orden de arranque que imponen los health checks: primero `db` y `redis` llegan a *Healthy*, luego arranca `backend`, y cuando este es *Healthy* arranca `frontend`.

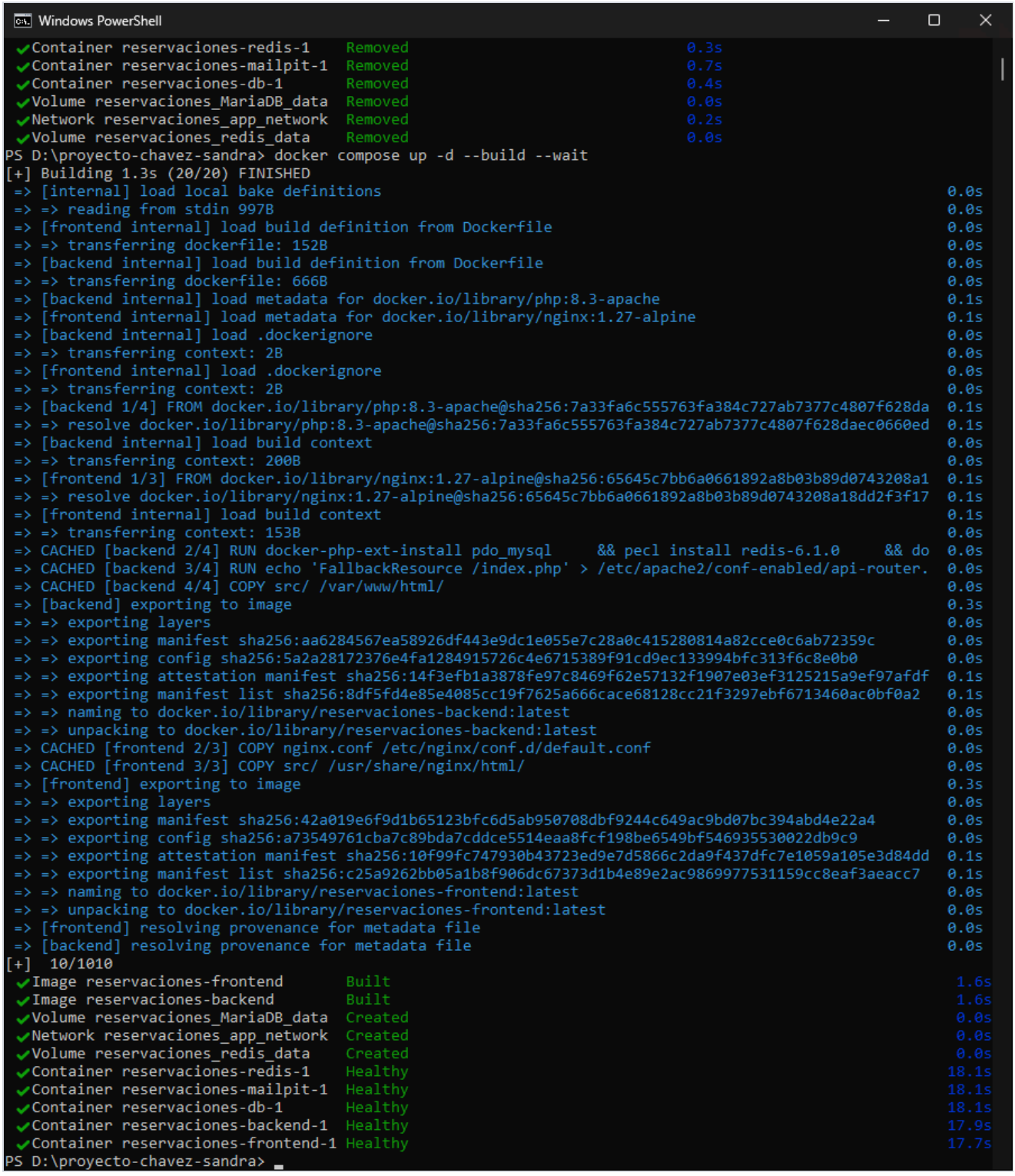


Figura 4. Reinicio limpio y construcción de las imágenes propias.

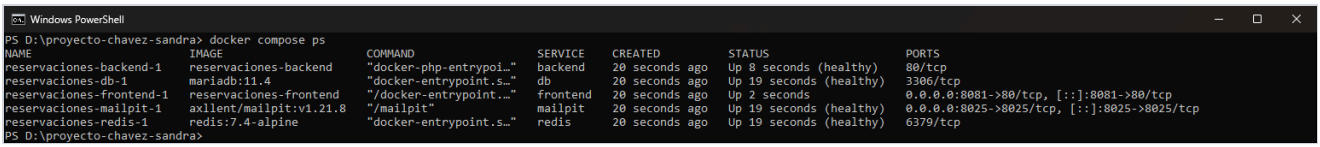


Figura 5. `docker compose ps`: los cinco servicios están activos y db, redis, backend y mailpit aparecen como *healthy*. Solo frontend (8081) y mailpit (8025) tienen puertos publicados.

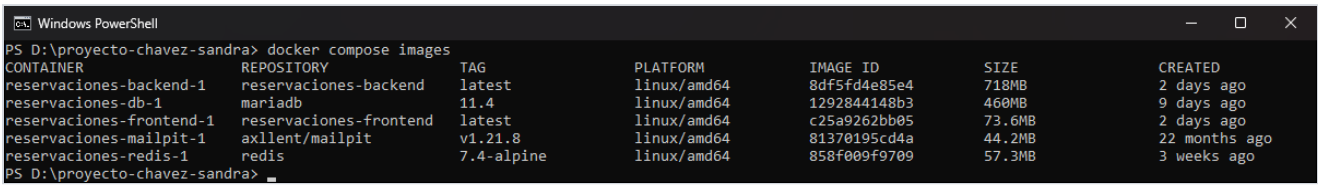


Figura 6. `docker compose images`: dos imágenes propias (frontend y backend) y tres imágenes oficiales con versión explícita.

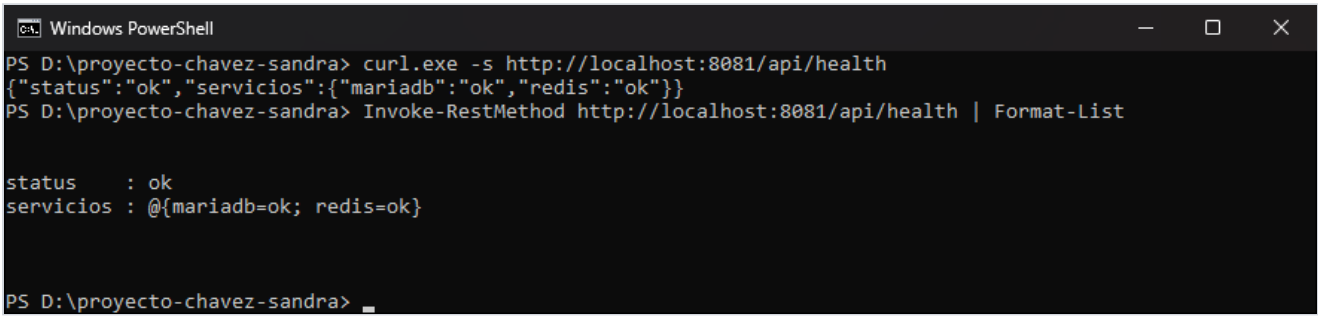


Figura 7. El endpoint `/api/health`, consultado con `curl.exe` y con `Invoke-RestMethod`, confirma la conexión con MariaDB y Redis a través del proxy de Nginx.

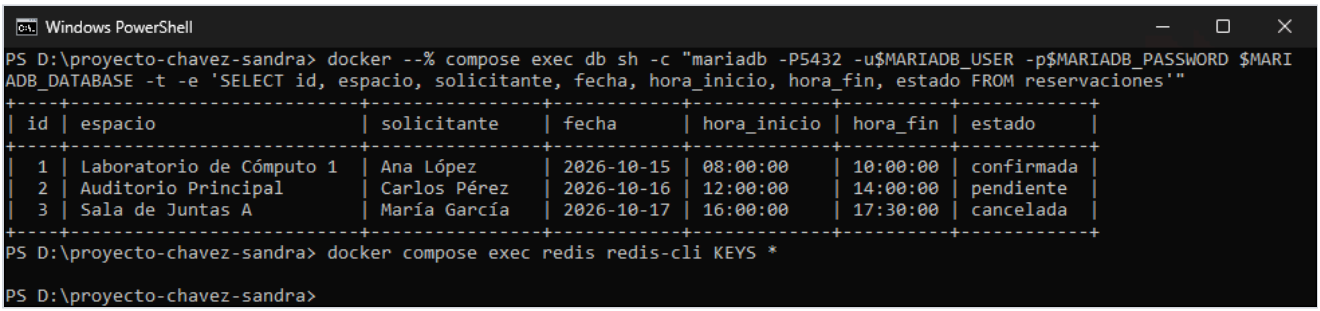


Figura 8. MariaDB contiene los tres registros de `init.sql`. Redis todavía no tiene claves.

7. Pruebas funcionales del CRUD

7.1 Consultar

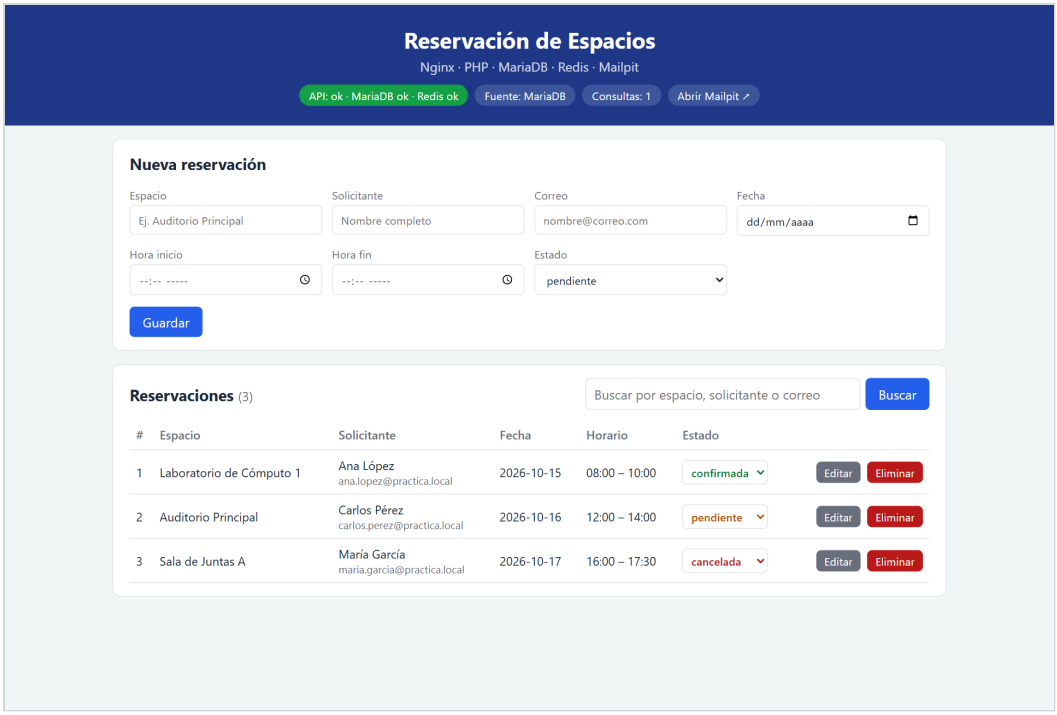


Figura 9. La interfaz en `http://localhost:8081` muestra los registros de MariaDB. Los indicadores superiores muestran el estado de la API, la fuente de los datos y el contador.

7.2 Crear

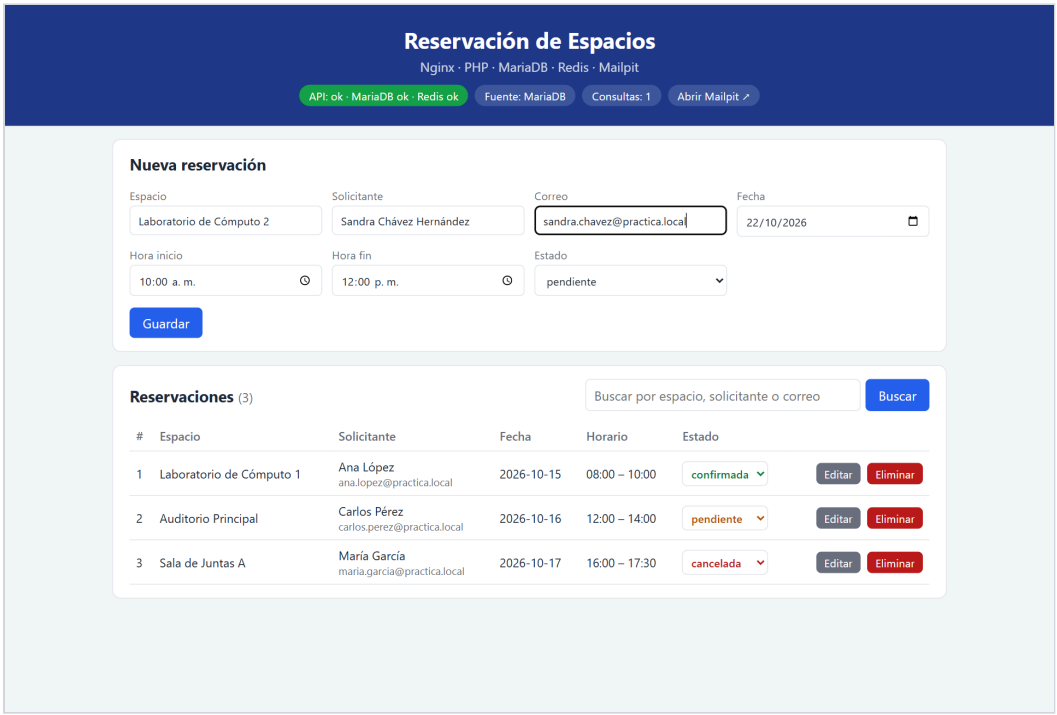


Figura 10. Formulario para registrar una nueva reservación.

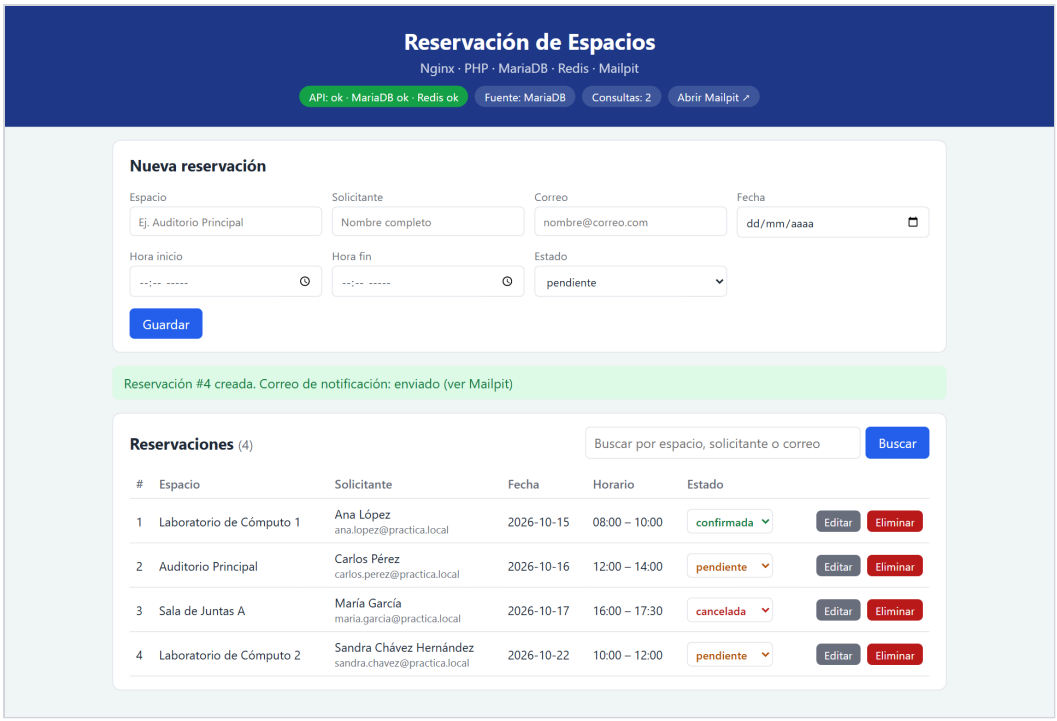


Figura 11. La reservación #4 se creó: hay un mensaje de éxito, el correo se envió y el listado se actualizó.

7.3 Manejo de errores

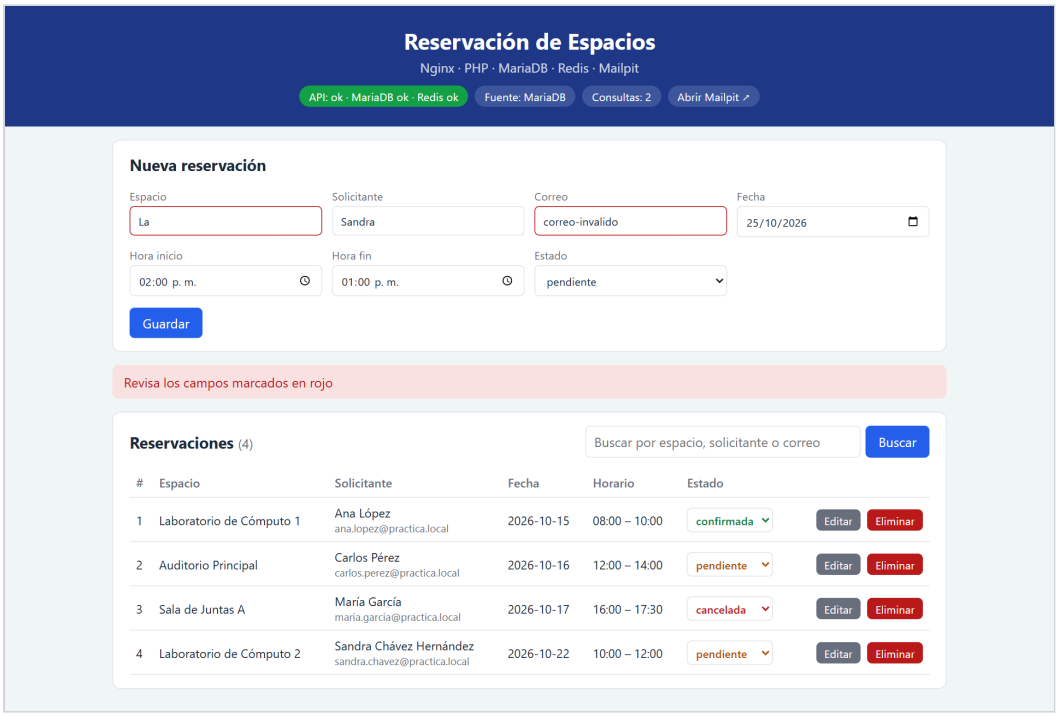


Figura 12. Validación en el frontend: los campos inválidos se marcan en rojo (espacio muy corto, correo inválido y hora de fin anterior a la de inicio).

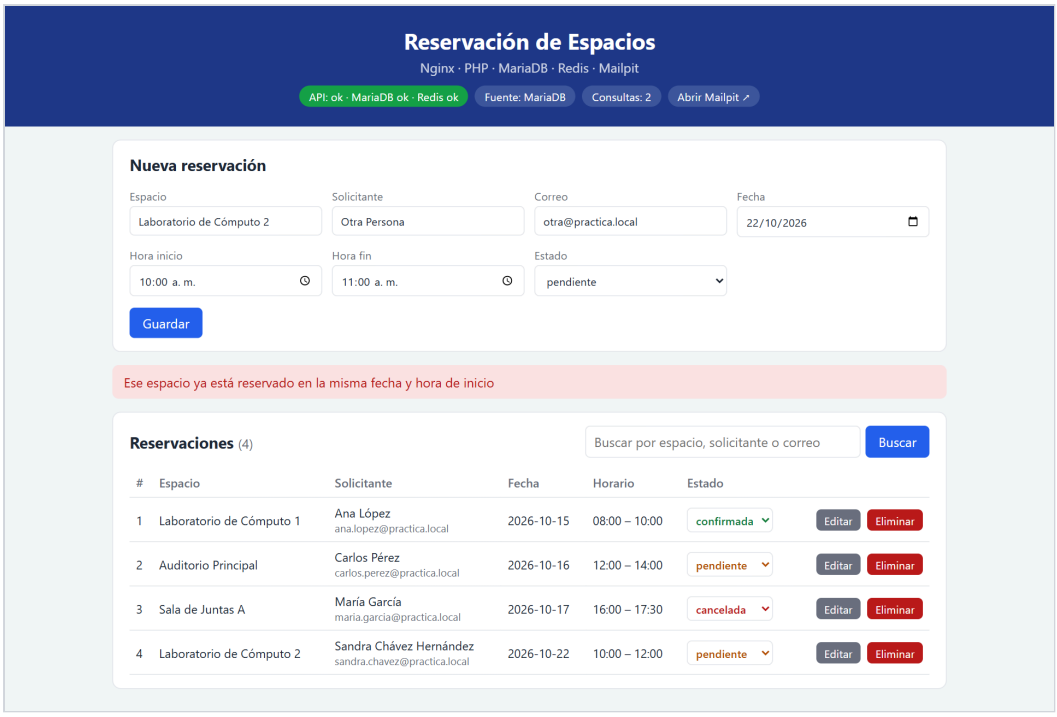


Figura 13. Al intentar reservar el mismo espacio, fecha y hora, el backend responde 409 y el mensaje aparece en la interfaz.

7.4 Actualizar

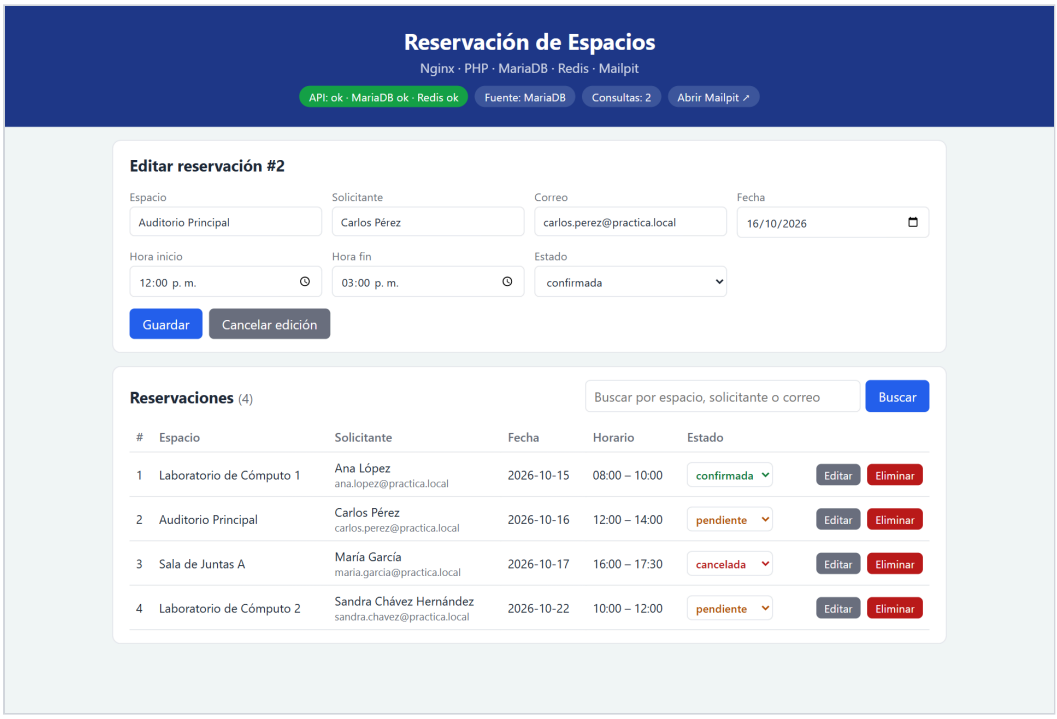


Figura 14. Edición de la reservación #2: se cambian la hora de fin (15:00) y el estado.

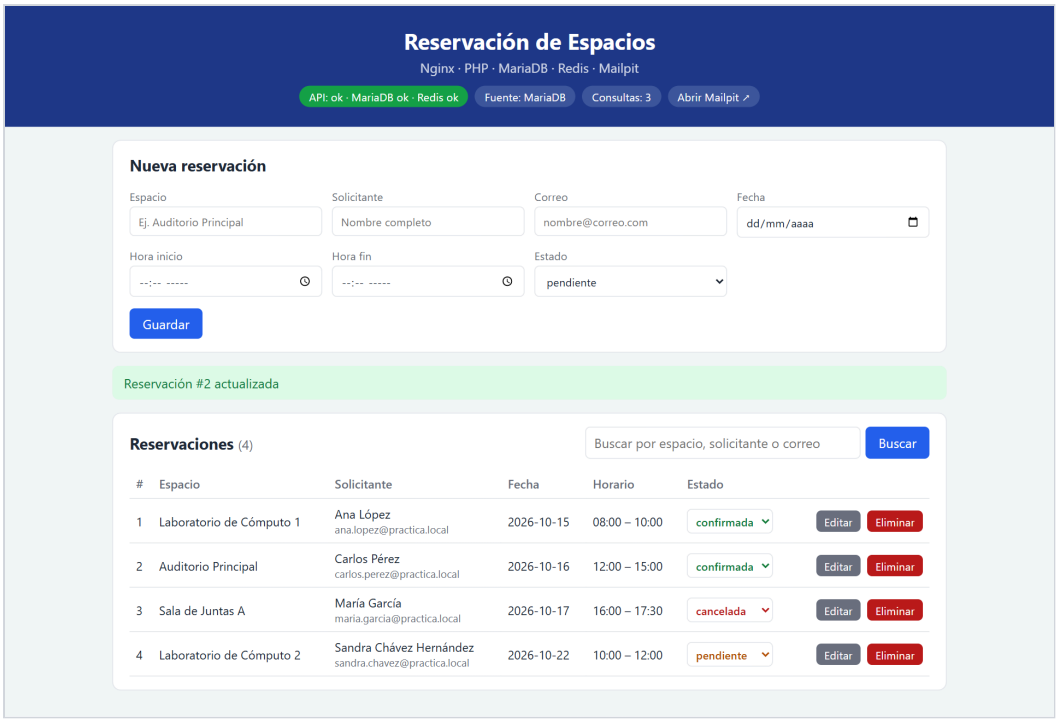


Figura 15. La reservación #2 se actualizó con el nuevo horario y el estado *confirmada*.

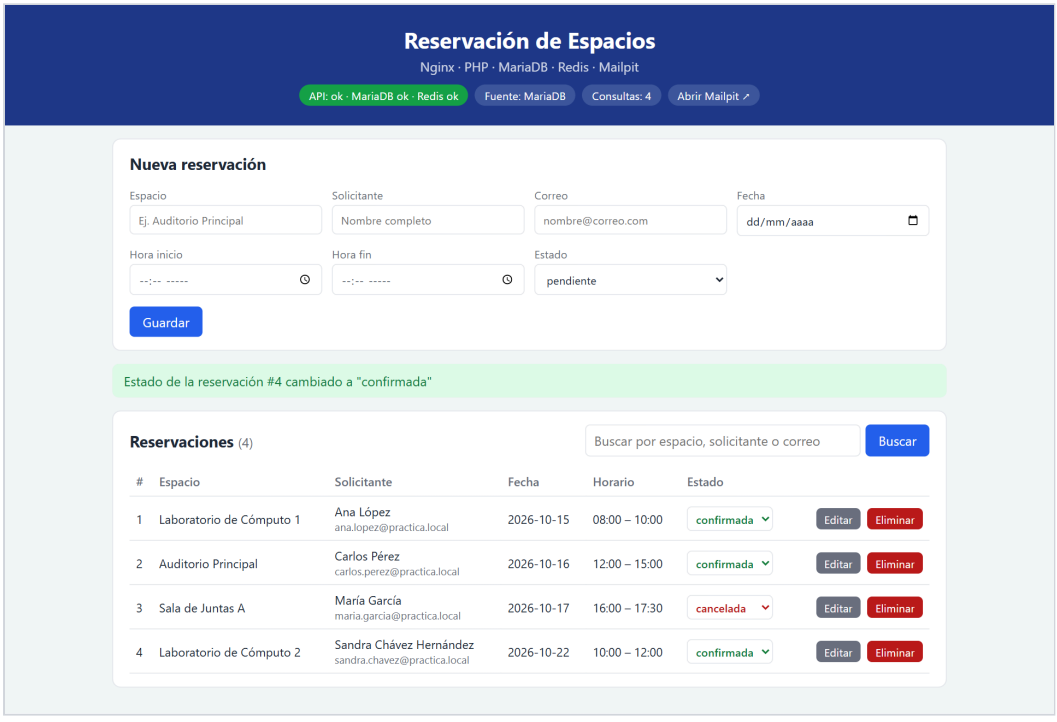


Figura 16. Cambio rápido del estado desde la tabla (reservación #4 pasa a *confirmada*).

7.5 Eliminar

El botón **Eliminar** pide confirmación con el mensaje *"¿Eliminar la reservación #3 (Sala de Juntas A)?"*. Solo si el usuario acepta se envía el `DELETE`.

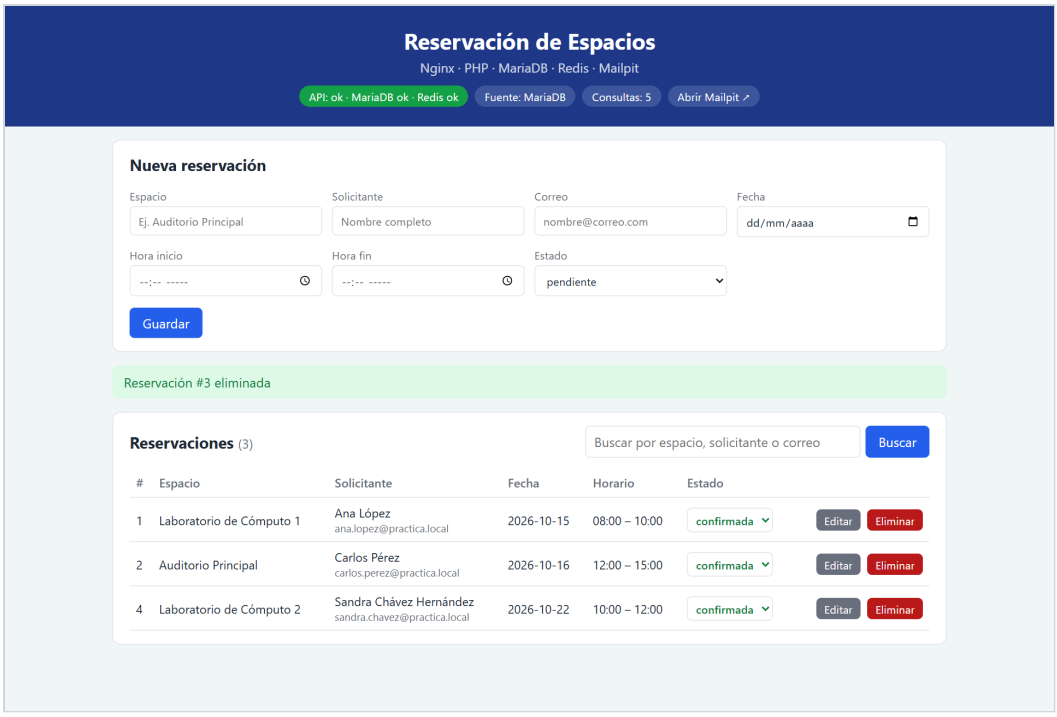


Figura 17. La reservación #3 se eliminó después de confirmar.

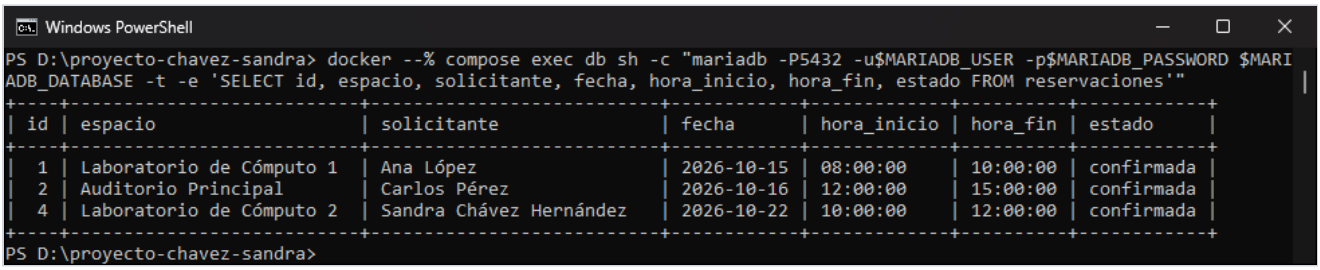


Figura 18. Consulta directa a MariaDB: los cambios hechos desde la interfaz quedaron guardados (#2 editada, #3 eliminada, #4 creada).

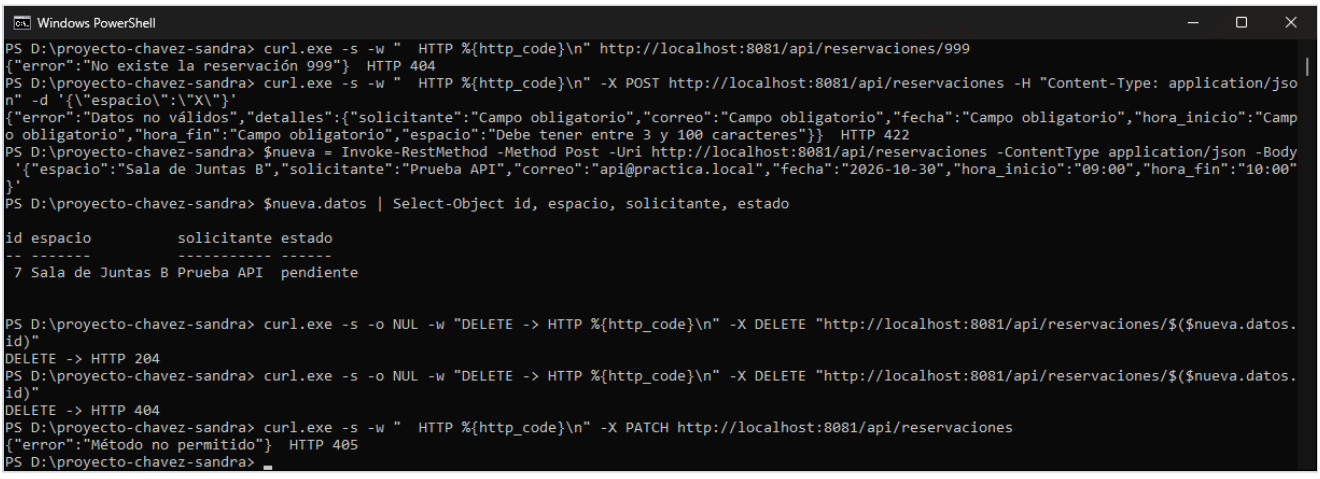


Figura 19. Códigos HTTP de la API: 404 (no existe), 422 (datos inválidos), creación de un registro de prueba, 204 (eliminado), 404 (ya eliminado) y 405 (método no permitido).

8. Persistencia en MariaDB

Se consultaron los registros, se eliminó el contenedor de la base de datos con `docker compose rm -sf db` sin borrar el volumen y se creó un contenedor nuevo. Los registros siguen presentes porque el directorio de datos de MariaDB está en el volumen nombrado `reservaciones_MariaDB_data`, que existe fuera del ciclo de vida del contenedor.

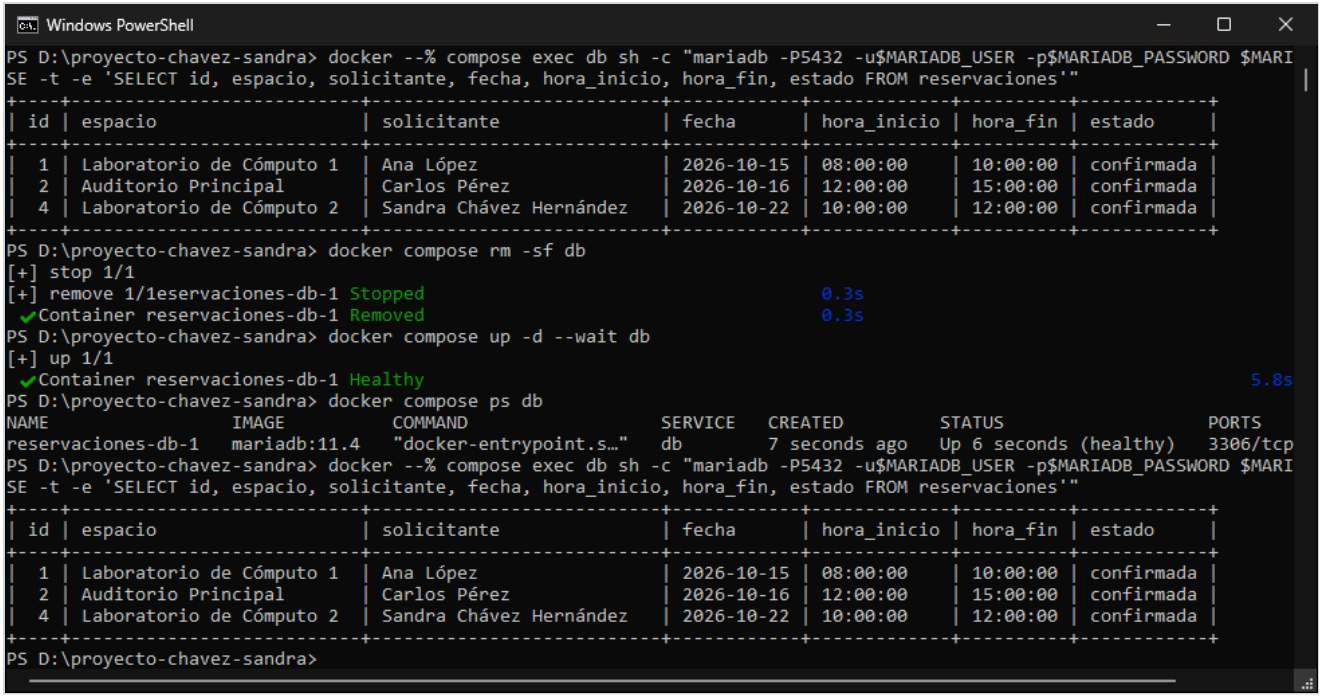


Figura 20. Los mismos registros aparecen antes y después de recrear el contenedor `db`.

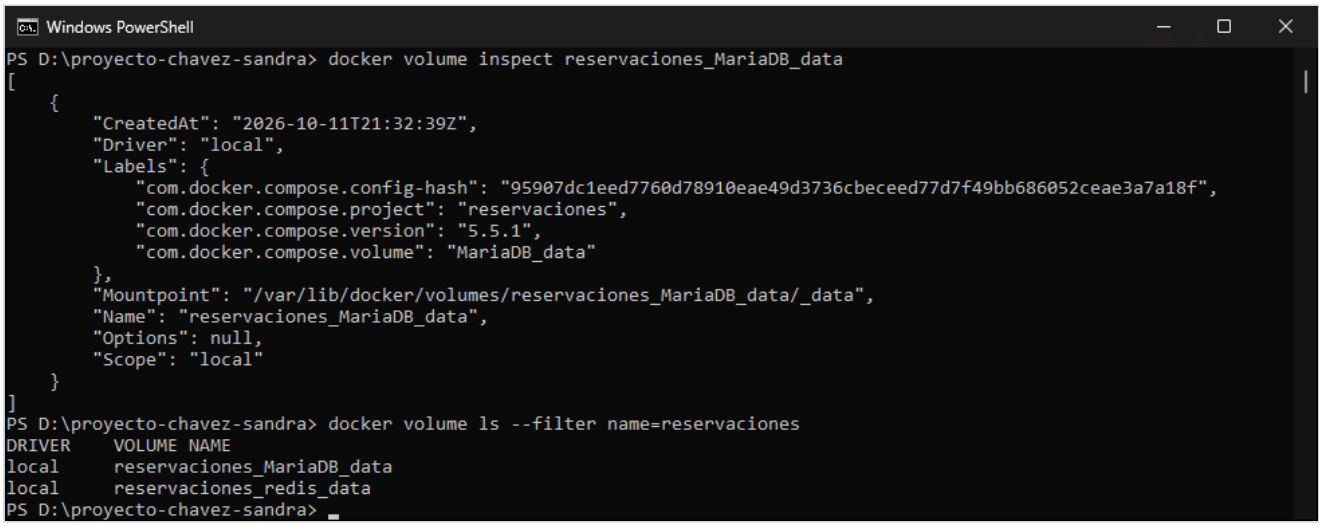


Figura 21. `docker volume inspect` del volumen de MariaDB, que pertenece al proyecto *reservaciones*.

9. Redis: caché, invalidación y contador

El listado se guarda en la clave `reservaciones:lista` con un TTL de 60 segundos (el mínimo pedido es 30). La primera consulta después de un cambio es un **MISS**: va a MariaDB y guarda el resultado en Redis. Las consultas siguientes son un **HIT**: se responden desde Redis. Cualquier creación, edición o eliminación borra la clave.

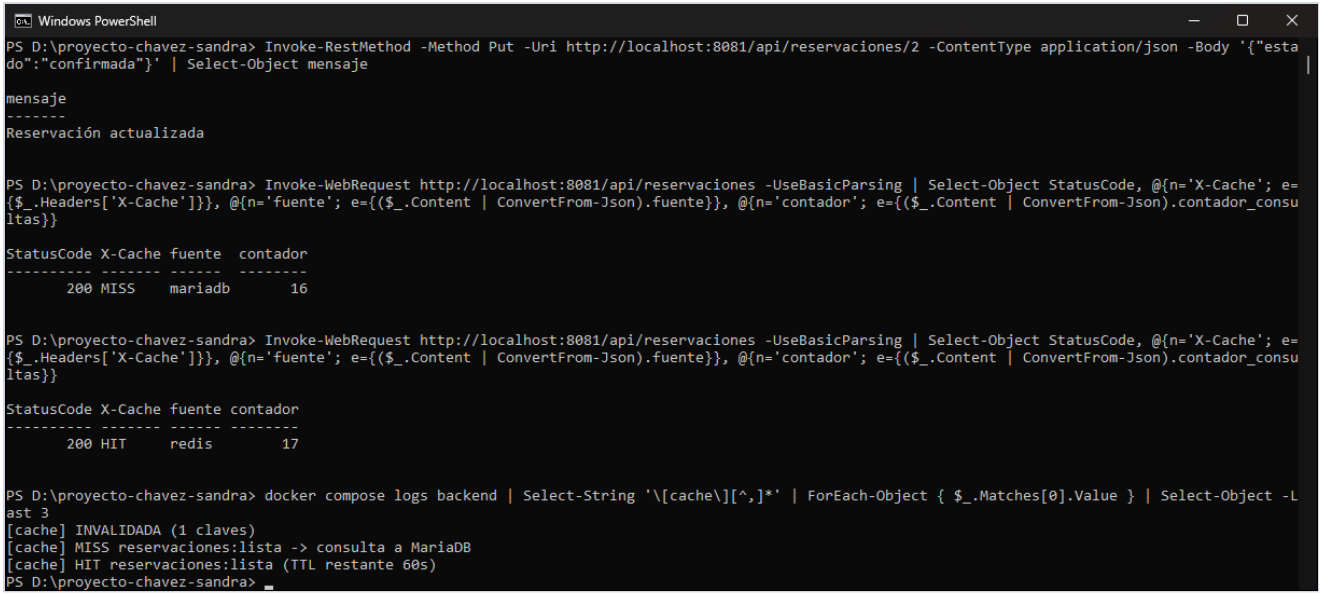


Figura 22. Después de un PUT (que invalida la caché), la primera consulta responde `X-Cache: MISS` desde MariaDB y la segunda `X-Cache: HIT` desde Redis. Los logs del backend registran INVALIDADA, MISS y HIT.



Figura 23. En la interfaz, el indicador "Fuente: Redis (caché)" aparece cuando el listado se sirve desde la caché.

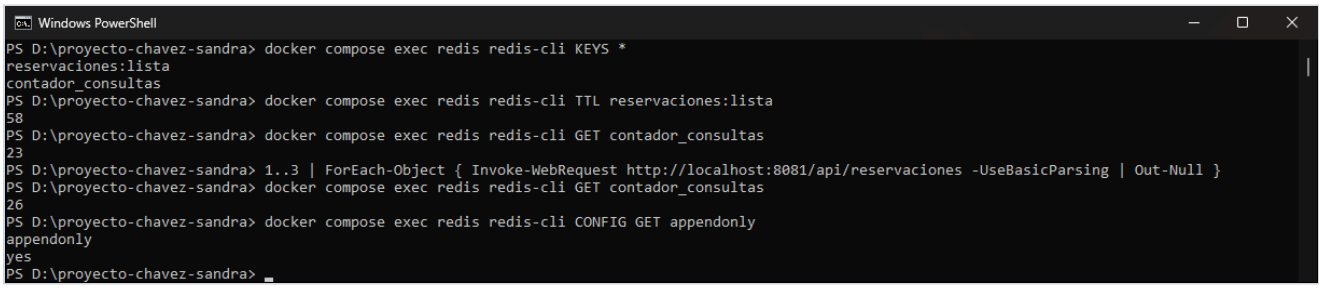


Figura 24. Claves de Redis, TTL restante, contador `contador_consultas` (sube de 23 a 26 tras tres consultas) y persistencia AOF activada.

10. Notificaciones con Mailpit

Cada vez que se crea una reservación, el backend envía un correo a `mailpit:1025`. El mensaje se puede ver en `http://localhost:8025` con su asunto, destinatario y contenido.

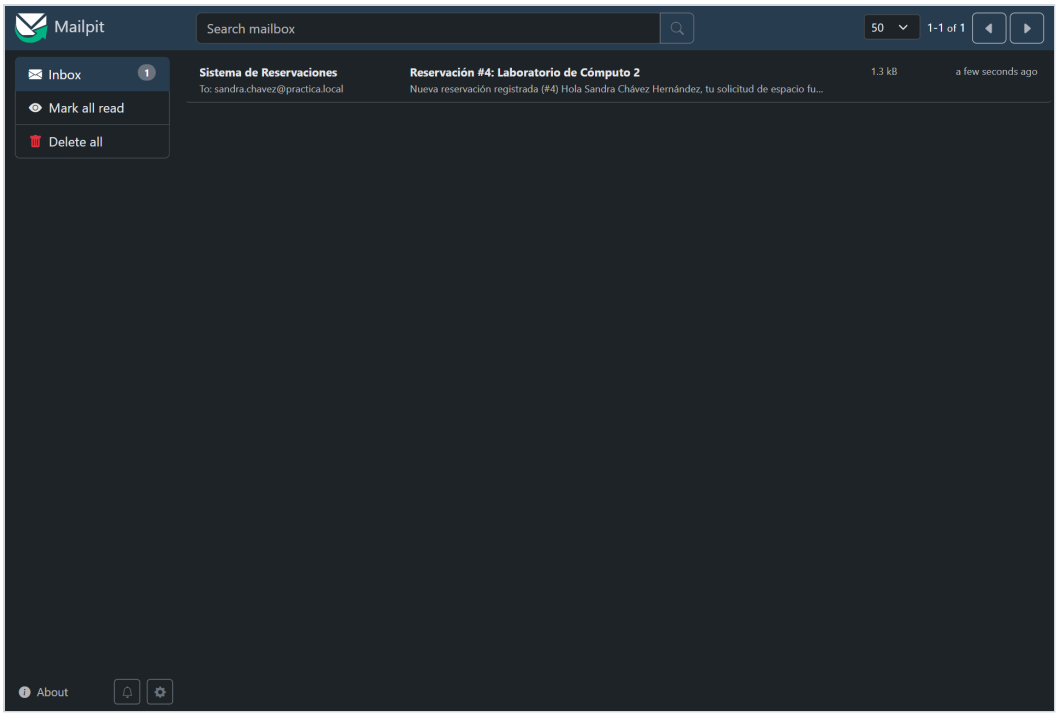


Figura 25. Bandeja de Mailpit con la notificación de la reservación creada.

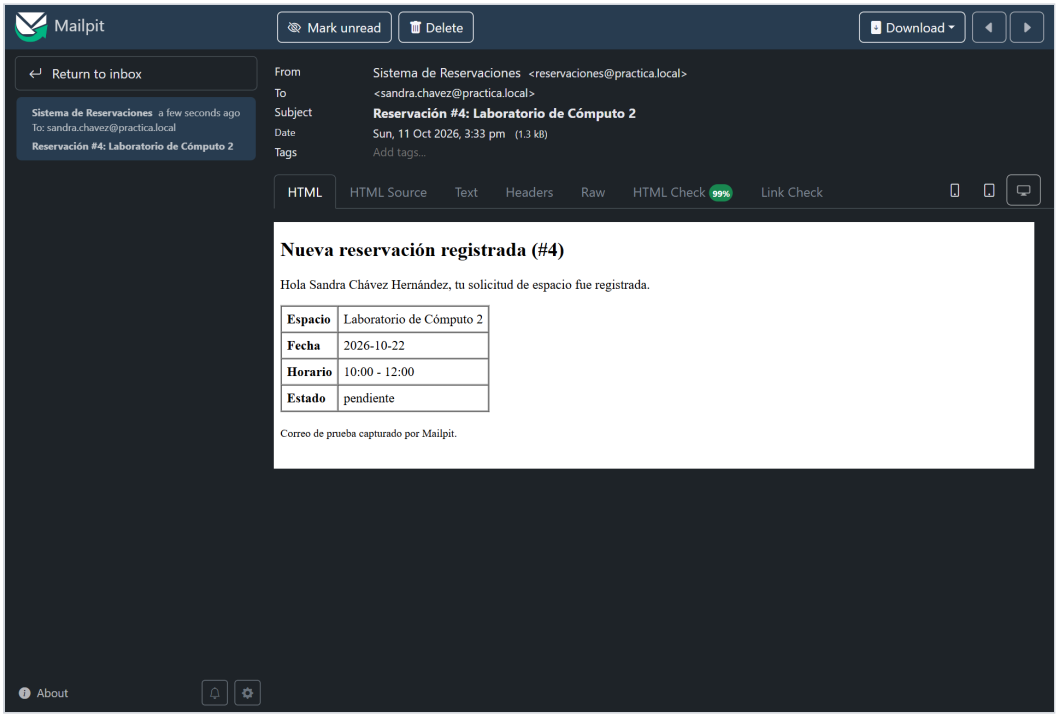


Figura 26. Detalle del correo: remitente, destinatario `sandra.chavez@practica.local`, asunto *"Reservación #4: Laboratorio de Cómputo 2"* y una tabla con los datos del registro.

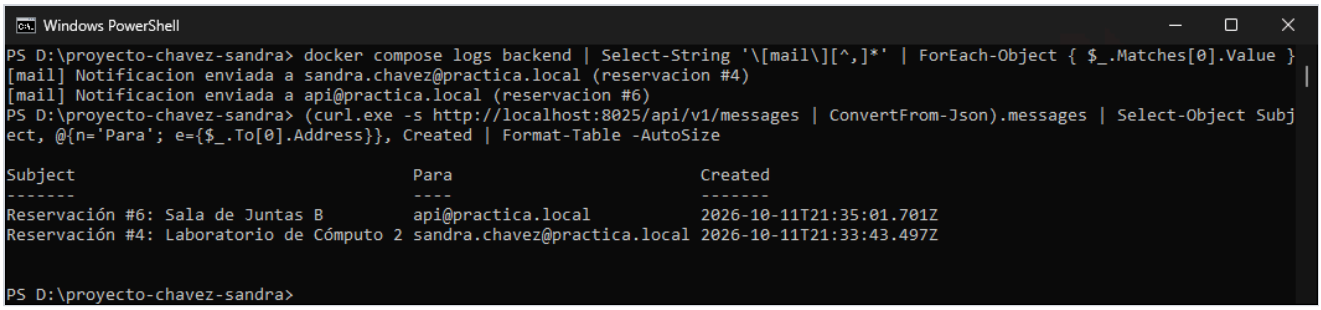


Figura 27. Logs del backend con los envíos y la lista de asuntos obtenida de la API de Mailpit.

11. Red, aislamiento y recuperación

11.1 Red app_network

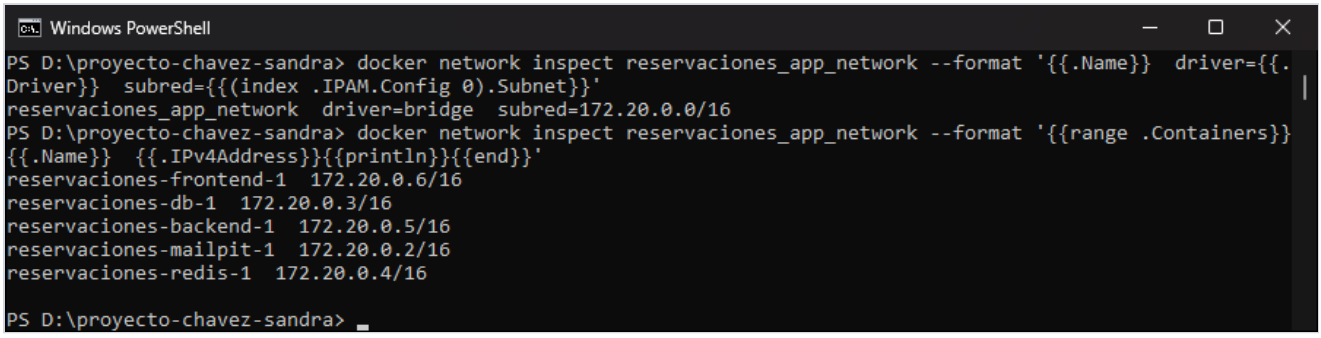


Figura 28. `docker network inspect`: los cinco contenedores están en `reservaciones_app_network` (bridge, 172.20.0.0/16).

11.2 Aislamiento

Solo `frontend` y `mailpit` publican puertos. En la columna PORTS, `db`, `redis` y `backend` no tienen el prefijo `0.0.0.0:`. El valor `3306/tcp` que aparece en db es solo el puerto declarado en la imagen de MariaDB, y no está publicado.

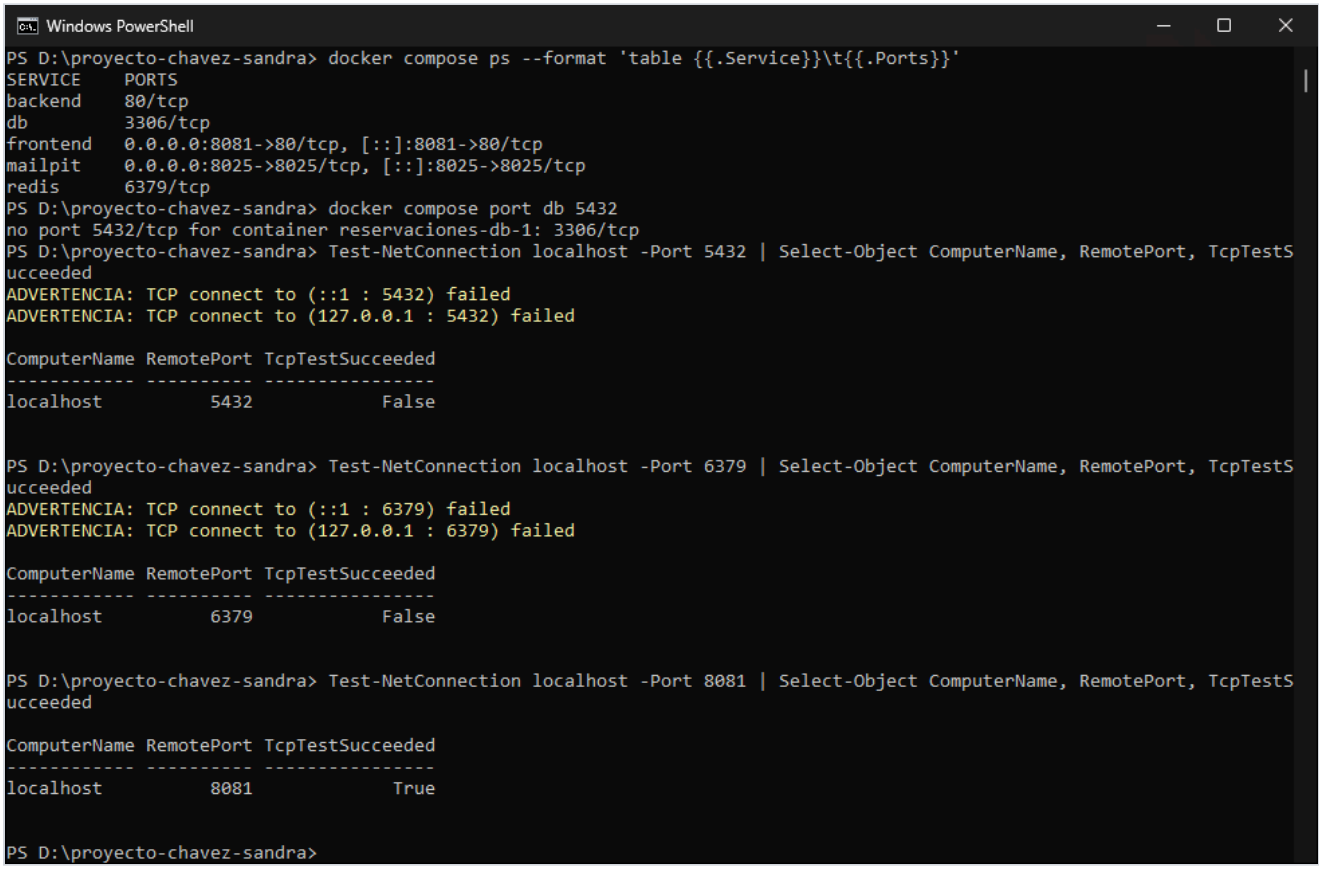


Figura 29. db y redis no tienen puertos publicados. Desde el host, `Test-NetConnection` falla en 5432 y 6379, y solo tiene éxito en 8081.

11.3 Recuperación

Con el backend detenido, Nginx responde **502 Bad Gateway**. Al volver a iniciarlo, la aplicación recupera su operación sin reiniciar Nginx, porque Nginx vuelve a resolver el nombre `backend` con el DNS interno.

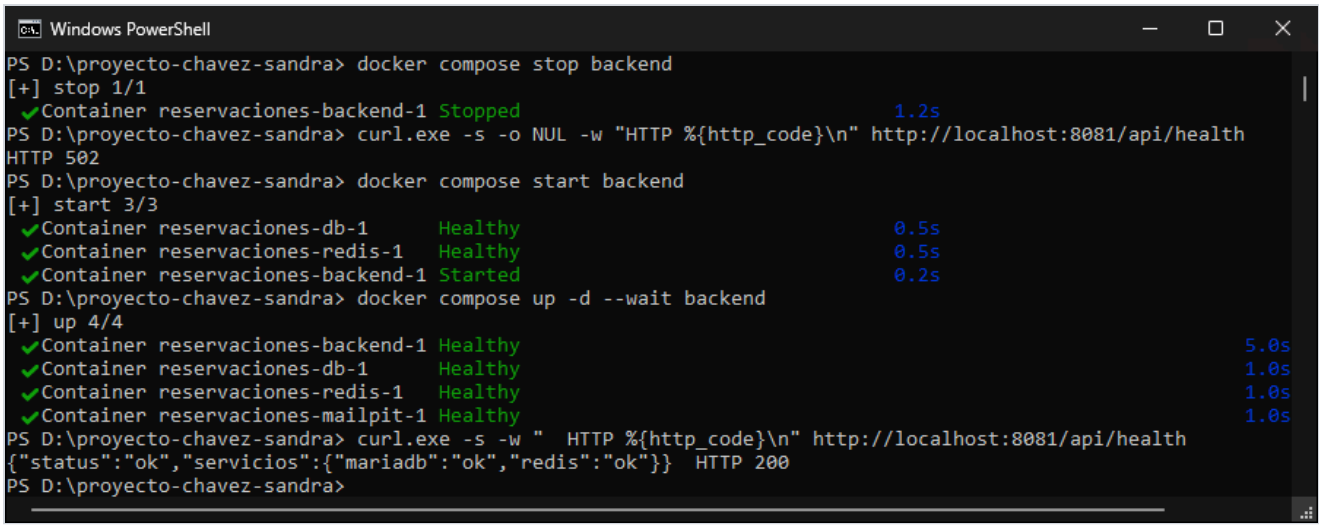


Figura 30. Detener el backend devuelve 502; al iniciarlo de nuevo, la API vuelve a responder 200.

12. Resolución de nombres por DNS interno

Docker Compose crea la red de usuario `reservaciones_app_network` y conecta a ella los cinco contenedores. Cada red de usuario tiene un servidor DNS interno en `127.0.0.11`, que aparece como `nameserver` en el `/etc/resolv.conf` de cada contenedor. Ese servidor registra el **nombre de cada servicio** como alias de la IP de su contenedor:

- **db**: el backend se conecta a `db:5432` y el DNS devuelve la IP del contenedor de MariaDB (172.20.0.2).
- **redis**: el backend se conecta a `redis:6379` y obtiene 172.20.0.3.
- **mailpit**: el backend envía SMTP a `mailpit:1025` y obtiene 172.20.0.4.
- **backend**: Nginx reenvía `/api/` a `http://backend:80` y obtiene 172.20.0.5.

Las IP pueden cambiar cuando un contenedor se recrea, pero el nombre se mantiene. Por eso la configuración usa nombres de servicio y nunca IP fijas ni `localhost`. Dentro de un contenedor, `localhost` es el mismo contenedor.

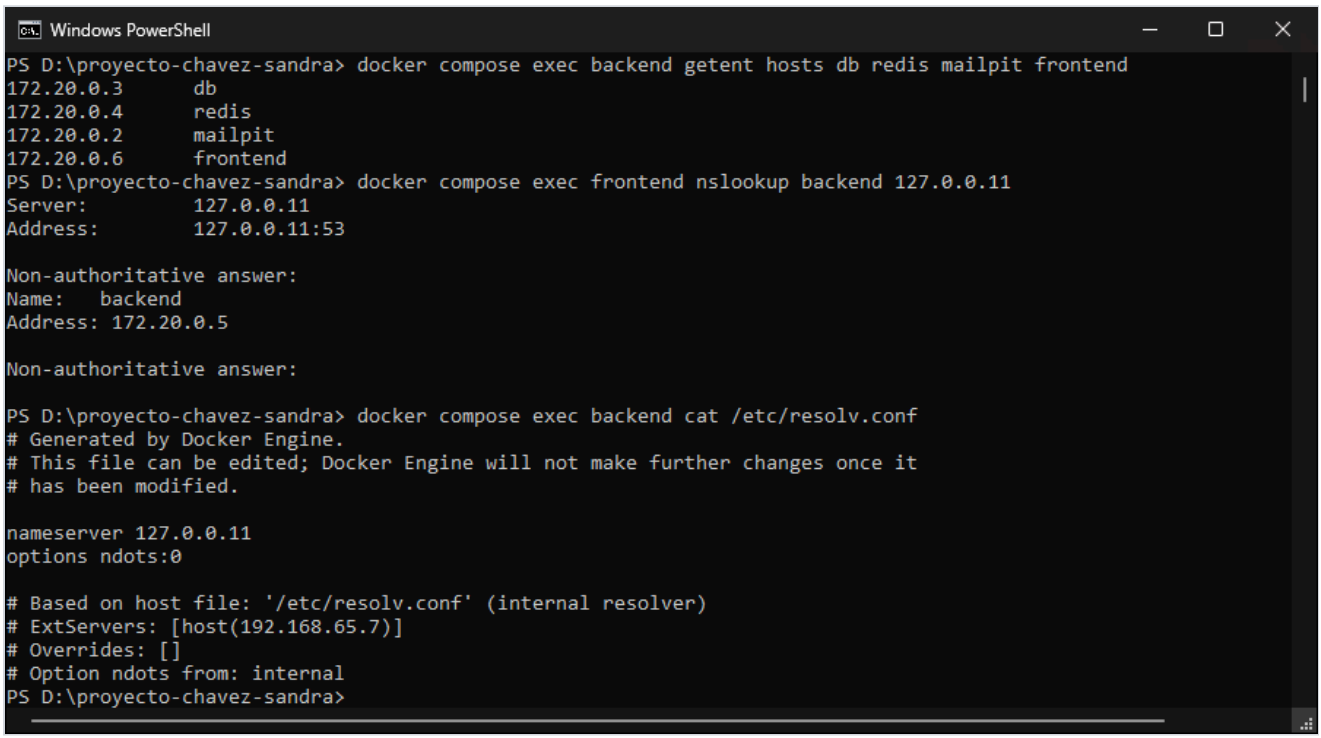


Figura 31. Resolución de `db`, `redis`, `mailpit` y `frontend` desde el backend, y de `backend` desde el frontend, con el DNS 127.0.0.11.

13. Seguridad

- Las contraseñas están solo en `.env`, que está excluido en `.gitignore`. En el repositorio se incluye `.env.example` sin valores reales.
- Redis requiere contraseña (`--requirepass`). El backend usa un usuario de aplicación de MariaDB, no el usuario root.
- No se usa `privileged`, no se monta `/var/run/docker.sock` y no se usa `container_name`, lo que permite escalar servicios.
- Solo se publican los puertos 8081 y 8025. MariaDB, Redis y el backend son accesibles únicamente por la red interna.
- La carpeta `lib/` del backend no se puede abrir por HTTP (responde 403). Las consultas usan sentencias preparadas (PDO), lo que evita inyección SQL. El frontend escapa el HTML de los datos.



Figura 32. `.gitignore`, `.env.example` sin secretos reales, ausencia de opciones peligrosas en `compose.yaml` y acceso denegado a `lib/`.

14. Mejora individual: filtro de búsqueda

Se agregó un campo de búsqueda en la interfaz. Envía `GET /api/reservaciones?buscar=texto`, y el backend busca el texto en el espacio, el solicitante y el correo con `LIKE` y parámetros preparados. Cada búsqueda se guarda en caché con su propia clave (`reservaciones:buscar:<md5>`), y esas claves también se borran cuando hay cambios. Involucra al frontend, al backend, a MariaDB y a Redis.

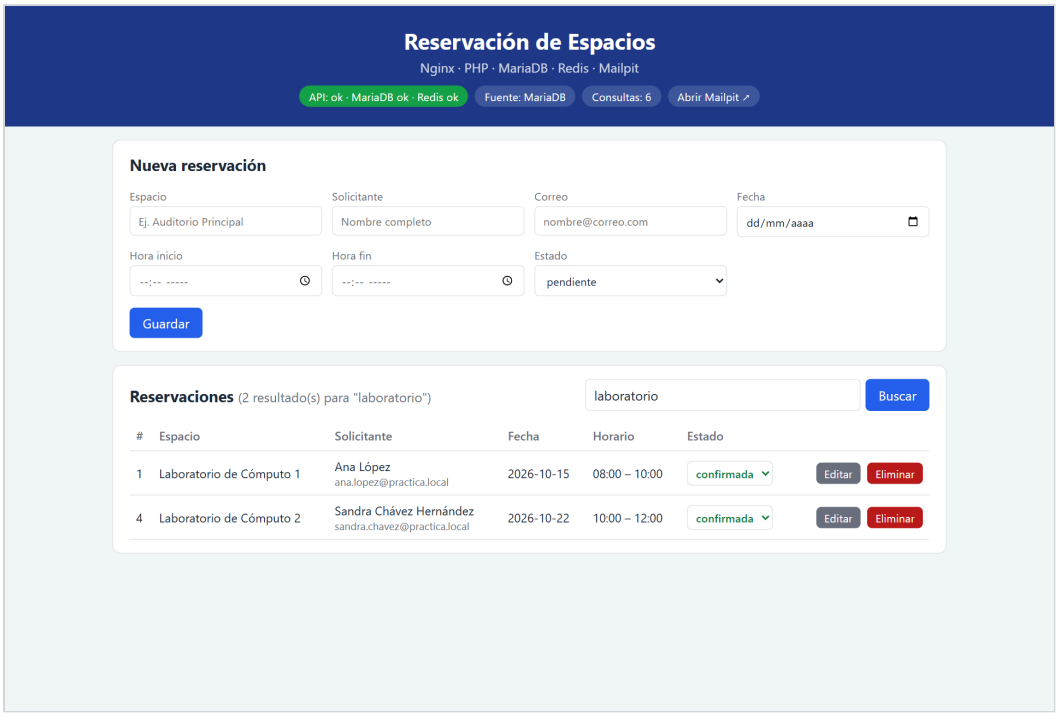


Figura 33. La búsqueda de "laboratorio" muestra solo las reservaciones que coinciden.

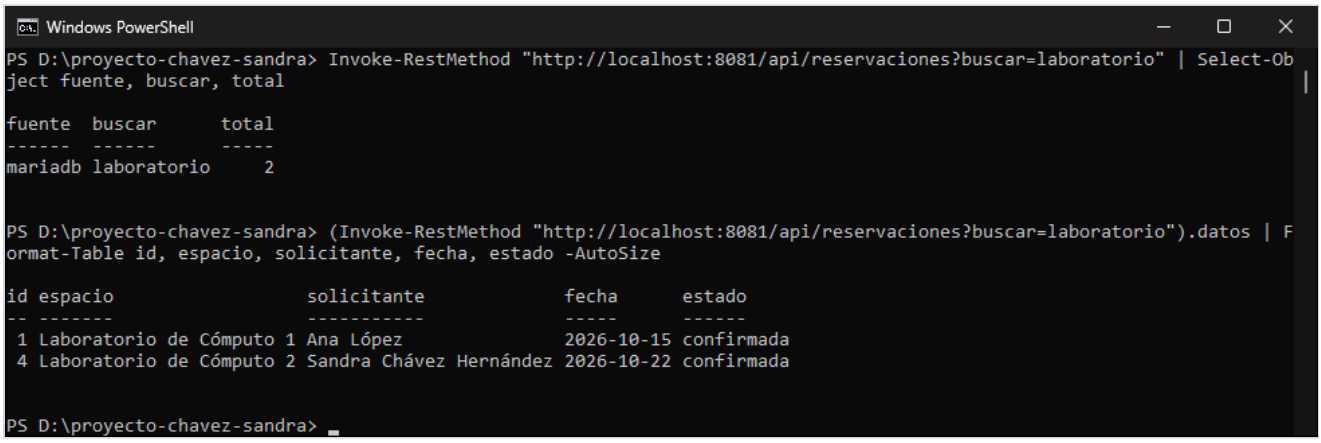


Figura 34. Respuesta de la API para `?buscar=laboratorio`.

15. Resumen de pruebas obligatorias

Prueba	Procedimiento	Resultado obtenido	Evidencia
Validación YAML	`docker compose config`	**Sin errores**	Fig. 3
Estado	`docker compose ps`	**5 servicios activos; db y redis healthy**	Fig. 5
CRUD	Crear, editar y eliminar desde la interfaz	**Cambios visibles y persistentes**	Fig. 9–19
Persistencia	Recrear db sin eliminar el volumen	**Los registros permanecen**	Fig. 20
Caché	Consultar dos veces y revisar logs y TTL	**2.ª consulta desde Redis (HIT)**	Fig. 22–24
Invalidación	Crear o editar un registro y consultar	**El listado muestra el dato nuevo (MISS)**	Fig. 11, 22
Contador	Consultar el listado varias veces	**El contador sube de 23 a 26**	Fig. 24
Correo	Crear un registro y abrir Mailpit	**Mensaje visible en el puerto 8025**	Fig. 25–27
Aislamiento	Revisar `docker compose ps`	**db y redis sin puertos publicados**	Fig. 29
Recuperación	Detener el backend y volver a iniciarlo	**502 → 200, la aplicación se recupera**	Fig. 30

16. Notas técnicas

> **Imagen, puerto y ruta de MariaDB.** La guía pide la imagen `MariaDB:16-alpine`, el puerto `5432` y la ruta `/var/lib/MariaDB/data`. Esos valores corresponden a PostgreSQL: MariaDB no tiene versión 16, su puerto por defecto es 3306 y su directorio de datos es `/var/lib/mysql`. Para cumplir con lo pedido de forma funcional:
>
> - Se usó `mariadb:11.4`, la versión LTS vigente, con versión explícita.
> - MariaDB se inició con `--port=5432` para respetar `DB_PORT=5432`.
> - El volumen nombrado `MariaDB_data` se montó en `/var/lib/mysql`, que es el directorio real de datos.

> El puerto del frontend se puede cambiar con la variable `FRONTEND_PORT` (por defecto 8081), por si otro proyecto ya ocupa ese puerto. Las capturas de terminal se tomaron de una consola real de Windows PowerShell, ejecutando cada comando dentro del directorio del proyecto.

17. Preguntas de reflexión

1. ¿Por qué el backend debe usar db:5432 y no localhost para conectarse a MariaDB?
Cada contenedor tiene su propio espacio de red. Dentro del contenedor del backend, `localhost` es el propio backend, donde no hay ninguna base de datos. `db` es el nombre del servicio, y el DNS interno de Docker lo convierte en la IP del contenedor de MariaDB dentro de `app_network`.

2. ¿Cuál es la diferencia entre ports y expose en Docker Compose?
`ports` publica un puerto del contenedor en la máquina anfitriona (por ejemplo, `8081:80`), así que se puede acceder desde fuera de Docker. `expose` solo documenta el puerto y lo deja disponible para otros contenedores de la misma red, sin publicarlo en el host.

3. ¿Qué información permanece cuando se elimina un contenedor y por qué?
Permanece lo que está en volúmenes (`MariaDB_data`, `redis_data`) o en bind mounts, porque se guarda fuera de la capa de escritura del contenedor. Lo que se escribió dentro del sistema de archivos del contenedor se pierde. Las imágenes tampoco cambian.

4. ¿Qué problema resuelve el health check antes de iniciar el backend?
Que un contenedor esté iniciado no significa que el servicio esté listo. MariaDB tarda unos segundos en inicializarse y ejecutar `init.sql`. Con `depends_on: condition: service_healthy`, el backend arranca cuando db y redis ya aceptan conexiones, y así se evitan fallos de conexión al iniciar.

5. ¿Cuándo puede una caché devolver información desactualizada y cómo lo evitaste?
Ocurre cuando los datos cambian en MariaDB mientras la copia en Redis sigue vigente. Para evitarlo, el backend invalida la caché en cada POST, PUT y DELETE: borra `reservaciones:lista` y las claves de búsqueda. Además, las claves tienen un TTL de 60 segundos como límite.

6. ¿Por qué Mailpit no debe considerarse un servidor de correo de producción?
Mailpit no entrega los mensajes a sus destinatarios: solo los captura para revisarlos. Tampoco ofrece, por defecto, autenticación, TLS, colas con reintentos, SPF/DKIM ni almacenamiento duradero. Está diseñado para pruebas en desarrollo.

7. ¿Qué servicio escalarías primero y qué cambio harías para evitar conflictos de puertos?
El **backend**, porque no guarda estado: el estado está en MariaDB y Redis. Para escalarlo, no debe publicar puertos en el host (en este proyecto usa `expose`) ni tener `container_name`. Así se puede ejecutar `docker compose up -d --scale backend=3`: Nginx sigue llamando a `backend` y el DNS de Docker reparte las peticiones entre las réplicas. Si se escalara un servicio con puerto publicado, habría que quitar el puerto fijo o usar un rango.

8. ¿Qué riesgos existirían si MariaDB y Redis se publicaran hacia Internet?
Ataques de fuerza bruta contra las contraseñas, robo, modificación o borrado de datos, explotación de vulnerabilidades conocidas, ransomware que borra bases expuestas, y abuso de Redis para escribir archivos o ejecutar comandos (`CONFIG SET`) o para consumir recursos (denegación de servicio). Por eso estos servicios se quedan en la red interna.

18. Lista de cotejo

Criterio	Cumple
El archivo compose.yaml pasa `docker compose config`.	✓
Los cinco servicios aparecen en `docker compose ps`.	✓
No se usó `container_name` en servicios que se pretenden escalar.	✓
El backend usa db, redis y mailpit como nombres de host.	✓
MariaDB conserva la información al recrear su contenedor.	✓
Redis muestra claves, TTL y contador relacionados con la aplicación.	✓
Mailpit recibe una notificación al crear un registro.	✓
El frontend consume `/api/` por medio de Nginx.	✓
El archivo .env no está versionado.	✓
El README permite reproducir el proyecto desde cero.	✓

Reproducir el proyecto desde cero

```
cd proyecto-chavez-sandra
Copy-Item .env.example .env   # editar las contraseñas
docker compose up -d --build
# Aplicación: http://localhost:8081    Mailpit: http://localhost:8025
```

19. Conclusiones

La práctica permitió desplegar una aplicación completa repartida en cinco contenedores con un solo archivo `compose.yaml`. La red interna y el DNS de Docker permiten que los servicios se encuentren por nombre sin depender de IP. El volumen nombrado separa los datos del ciclo de vida del contenedor, como se comprobó al recrear MariaDB sin perder información. Los health checks y `depends_on` garantizan un orden de arranque confiable.

Redis mostró una integración real: el listado se sirvió desde la caché, la caché se invalidó en cada cambio y el contador aumentó con cada consulta. Mailpit permitió verificar el envío de notificaciones sin enviar correo real. Por último, publicar solo los puertos necesarios y mantener las contraseñas fuera del repositorio reduce la superficie de ataque, una práctica básica para cualquier despliegue.